

Full-Stack AI Model

Retrieval-Augmented AI Knowledge & Context Engineering Platform

Author: Keeno Smith

Project Type: Full-Stack Software AI Application / SaaS-Style Web Platform

Platform: Web Application

Sector / Domain: Artificial Intelligence, Generative AI

Architecture: React + Node.js + Express + MongoDB + Semantic Retrieval _ LLM Inference

Primary AI Technologies: Groq LLM inference, Hugging Face Transformers, ONNX embeddings, Retrieval-Augmented Generation (RAG)

Frontend: React, Vite, React Markdown, syntax highlighting

Backend: Node.js, Express, Mongoose

Database: MongoDB

Document Processing: PDF, DOCX, Markdown, syntax highlighting

Development Model: Full-stack product development, API-driven architecture and iterative feature engineering.

Executive Summary

The Retrieval-Augmented AI Platform is a full-stack web application designed to combine conversational interaction with persistent context, configurable response behaviour, reusable knowledge, semantic document retrieval and streamed generative AI responses.

The platform moves beyond a basic prompt-and-response implementation by introducing several independent capabilities that work together as a single application:

- Persistent conversations
- Configurable conversational context
- Personality and response configuration
- Persistent memory
- User-managed knowledge
- Document ingestion
- Document chunking
- Local semantic embedding generation
- Vector similarity retrieval
- Retrieval-Augmented Generation
- Configurable model parameters
- Streaming AI responses
- Conversation search and management
- Response cancellation
- Error handling
- Responsive application workflows
- Persistent application state

The result is a technically substantial AI application in which the frontend, backend, database, AI inference layer, document-processing pipeline and retrieval system operate through defined application boundaries.

The project demonstrates breadth across **frontend engineering, backend/API development, AI application development, data modelling, information retrieval, product design, application architecture, QA, troubleshooting, maintenance and technical product ownership.**

Primary Objective

The primary objective was to develop a practical AI-powered knowledge and conversation platform capable of providing context-aware responses while allowing users to control the information supplied to the underlying language model.

Rather than treating every interaction as an isolated request, the platform establishes a persistent environment in which conversations, memory, personality configuration and external knowledge can contribute to the generation process.

- The technical objective therefore extends across four major areas:
- **Conversational intelligence**
Provide streamed natural-language generation through an LLM inference API.
- **Context management**
Allow conversations to incorporate selected historical messages, personality settings, memory and knowledge.
- **Knowledge retrieval**
Transform uploaded documents into searchable semantic representations and retrieve relevant content using vector similarity.
- **Product usability and lifecycle management**
Provide persistent conversation, memory and knowledge management through a complete web application rather than an isolated AI API.

The Business Problem

Generative conversational AI systems can provide powerful language generation, but their usefulness for recurring workflows is limited when users cannot maintain their own conversations, reusable knowledge and contextual preferences within a controlled application environment.

The platform addresses this problem by introducing a persistent application layer around generative AI.

The solution enables users to:

- Maintain conversations over time
- Control which contextual information is included
- Establish reusable response preferences
- Store reusable memory
- Upload and manage reference material
- Search knowledge semantically rather than relying exclusively on keywords
- Receive responses progressively through streaming
- Manage multiple conversations independently

This creates a foundation for knowledge-driven AI workflows where the value comes not only from the underlying language model, but also from the **context, data, retrieval and application architecture surrounding it**.

What The Application Does

At a functional level, the platform provides a conversational workspace backed by persistent application data and an AI retrieval pipeline.

A typical interaction follows the broader workflow:

Prompt → Context → Retrieval → AI Request → Model → Streamed Response

The application manages the surrounding context and sends the resulting message structure to the backend AI endpoint.

The backend validates the request, communicates with the Groq inference platform and converts the streamed model output into an application-consumable Server-Sent Events stream.

The frontend consumes that stream progressively, renders the response as Markdown, supports syntax-highlighted code and persists the completed conversation state.

Alongside conversations, the application provides dedicated management workflows for:

- Knowledge
 - Memories
 - Conversations
 - Context configuration
 - Personality configuration
 - Model configuration
-

Who It Serves

The platform is suited to users who need more control over conversational AI and reusable knowledge than a simple one-off prompt interface provides.

Potential users include:

- Software developers
- Technical professionals
- Researchers
- Knowledge workers
- Students
- Business users working with reference material
- Teams exploring AI-assisted knowledge workflows
- Users who require persistent conversational context

Its architecture also provides a foundation that could be extended into specialised enterprise knowledge systems, internal documentation platforms, technical research tools or domain-specific AI applications.

Overall Solution

The solution is structured as a full-stack web platform with distinct application responsibilities.

The **React frontend** manages the user-facing application state and contextual configuration.

The **Express backend** provides REST API boundaries for conversations, memories, knowledge and AI generation.

MongoDB provides persistence for conversations, messages, memories and processed knowledge.

The **knowledge pipeline** extracts uploaded documents, normalises their content, divides them into overlapping chunks and generates local semantic embeddings.

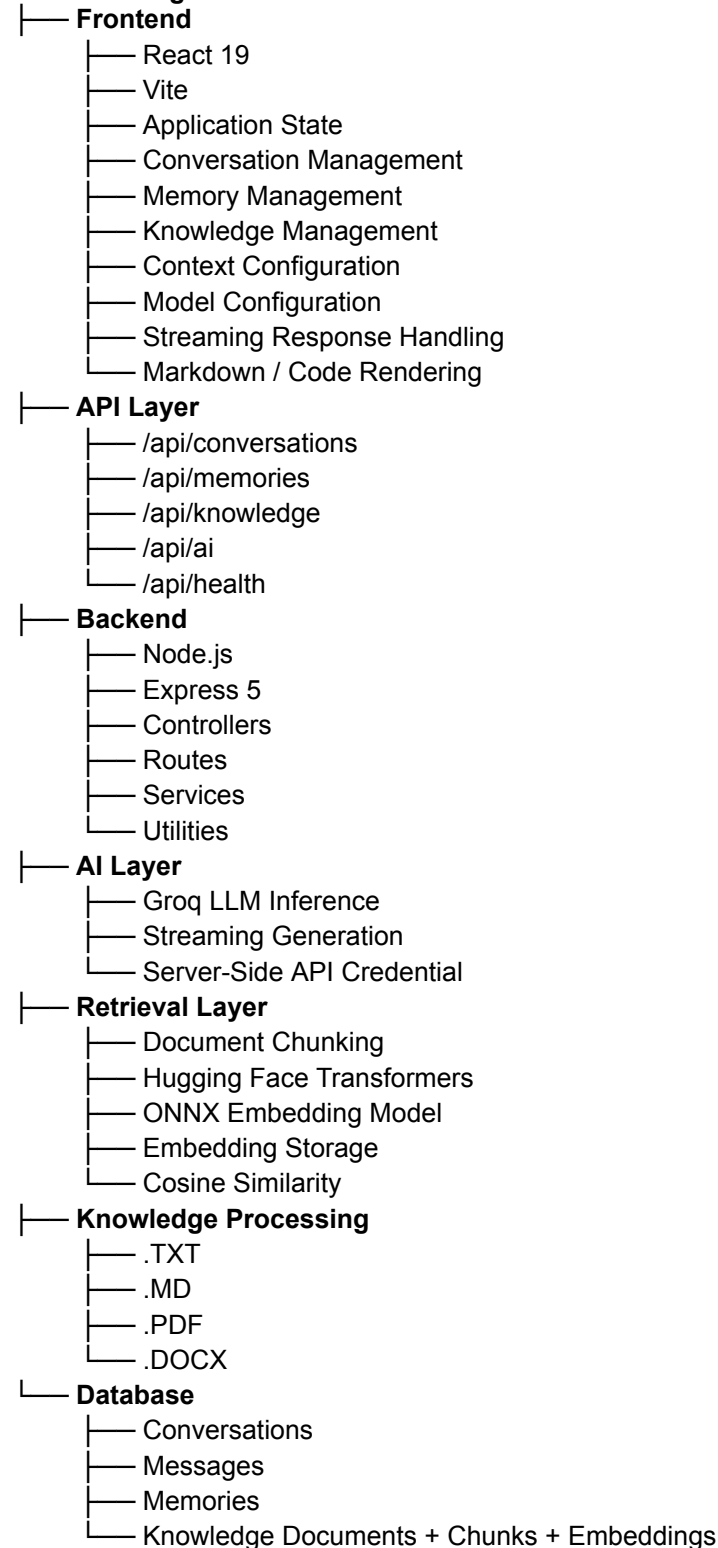
The **retrieval layer** generates an embedding for a user's search query, compares it against stored document embeddings using cosine similarity and returns the highest-scoring chunks.

The **LLM layer** uses Groq's API for streamed generative inference.

Together, these components establish a retrieval-augmented application architecture rather than a simple frontend connected directly to an AI provider.

Technical Architecture

Retrieval-Augmented AI Platform



Frontend Engineering

The frontend is built with **React 19** and **Vite**, using a modular API layer to separate HTTP communication from the primary application experience.

The central application layer coordinates:

- Conversation state
- Current conversation selection
- Message state
- Drafts
- Memory state
- Knowledge state
- Context configuration
- Personality configuration
- Model configuration
- Streaming state
- Response cancellation
- Search
- Editing
- Deletion
- Loading states
- Error states
- Responsive presentation

The frontend API modules provide dedicated boundaries for the major backend resources:

React Application

```
├── api/conversations.js
│   └── Conversation REST API
```

```
├── api/memories.js
│   └── Memory REST API
```

```
├── api/knowledge.js
│   └── Knowledge REST API
```

```
└── api/ai.js
    └── Streaming AI API
```

This separation keeps transport logic distinct from the broader application workflow.

The AI transport layer additionally implements streaming consumption using the Fetch API's `ReadableStream` interface and an `AbortController` signal, allowing an in-progress generation to be cancelled.

AI Response Streaming

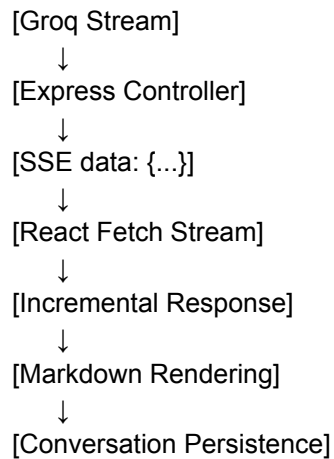
A particularly important engineering component is the end-to-end streaming pipeline.

The frontend sends the prepared message array to:

POST `/api/ai/chat`

The backend initiates a streamed Groq completion.

The backend then converts the provider stream into Server-Sent Events:



This architecture avoids waiting for the entire generated response before displaying it.

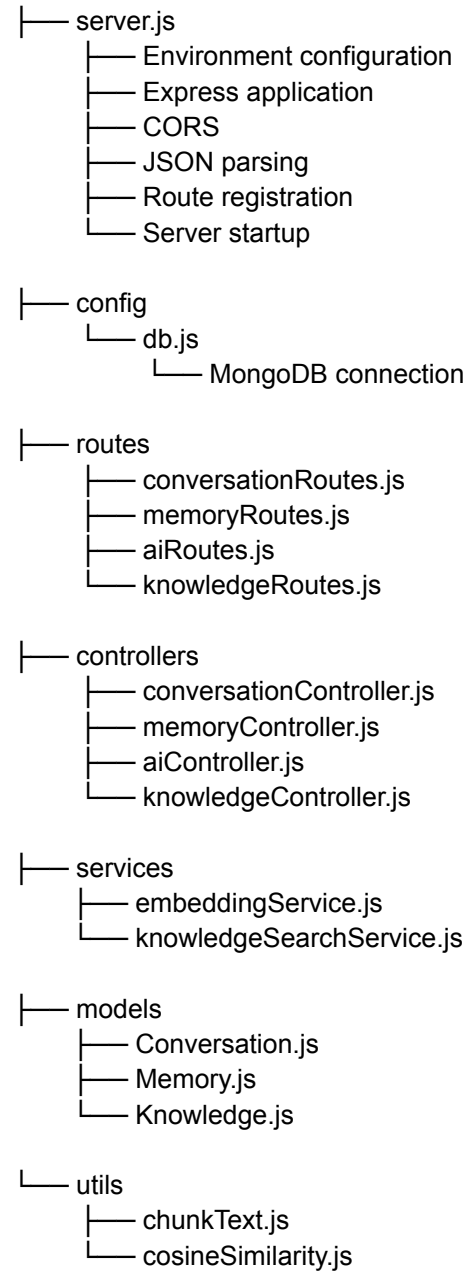
The frontend incrementally processes incoming chunks, reconstructs the complete response and handles the `[DONE]` termination event.

The implementation also accounts for errors occurring both before and after streaming headers have been sent, which is important because HTTP error handling changes once a streaming response has begun.

Backend Engineering

The backend uses **Node.js with Express 5** and follows a conventional route/controller structure.

Backend



This creates clear separation between:

HTTP routing → **controller behaviour** → **business/service processing** → **persistence and AI infrastructure**.

API Design

The platform exposes several resource-oriented API boundaries.

Conversations

/api/conversations

Supports creation, retrieval, updating, deletion and conversation management.

Conversation retrieval is ordered by most recently updated records, supporting a practical conversation-history workflow.

Memories

/api/memories

Provides persistent CRUD operations for reusable memory records.

Knowledge

/api/knowledge

Supports:

- Knowledge retrieval
- Knowledge creation
- Knowledge updates
- Knowledge deletion
- Document upload
- Semantic search

AI

/api/ai/chat

Provides streamed language-model generation.

Health

/api/health

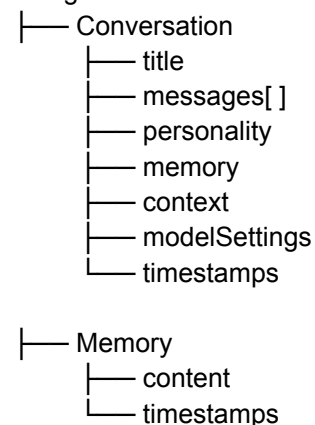
Provides a lightweight application health endpoint.

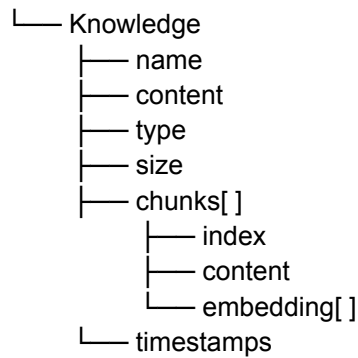
Database Architecture

MongoDB is used through **Mongoose** as the primary persistence layer.

The principal data models are:

MongoDB





The conversation model is particularly significant because it encapsulates multiple dimensions of conversational configuration.

It contains:

- Message history
- Personality configuration
- Memory configuration
- Context configuration
- Model configuration

The message schema also explicitly constrains message roles to:

user
assistant
system

This creates a structured representation of the conversation rather than storing an unstructured transcript.

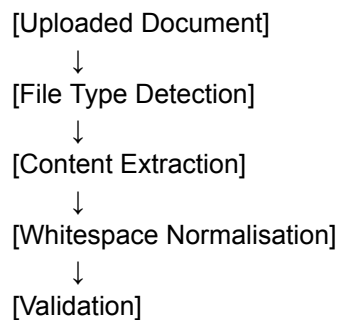
Knowledge Ingestion Pipeline

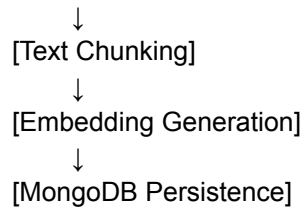
The knowledge subsystem provides an end-to-end document processing workflow.

Supported upload formats include:

- .txt
- .md
- .pdf
- .docx

The processing pipeline is:





PDF content is processed through **pdf-parse**, while DOCX content is extracted using **mammoth**.

Text and Markdown documents are handled as textual content.

A 10 MB upload limit is enforced through Multer's memory-storage configuration.

Chunking Strategy

Knowledge content is divided into overlapping character-based chunks.

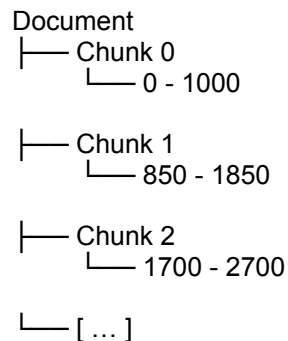
The default strategy uses:

Chunk size: 1,000 characters

Overlap: 150 characters

The overlap is important because relevant information can otherwise be divided across chunk boundaries.

The resulting structure is:



This provides a straightforward and transparent foundation for semantic retrieval.

Semantic Embedding Architecture

The platform uses the Hugging Face Transformers ecosystem with an ONNX-based embedding model:

`onnx-community/all-MiniLM-L6-v2-ONNX`

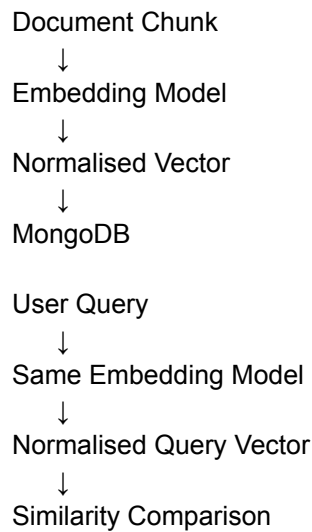
The embedding model is loaded lazily and cached in memory.

The embedding process uses:

- Feature extraction
- Mean pooling
- Normalisation

The same embedding mechanism is used for both document chunks and search queries, providing a consistent vector space for similarity comparison.

The architecture therefore follows:

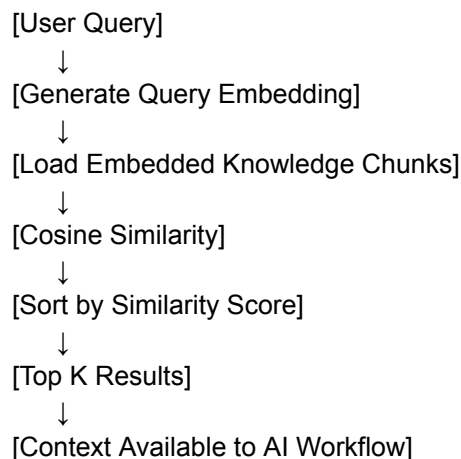


This is a meaningful AI engineering capability because the application incorporates a locally executed semantic representation layer rather than relying entirely on external AI APIs.

Retrieval-Augmented Generation

The knowledge subsystem implements a lightweight application-level RAG architecture.

The retrieval process is:



The default retrieval limit is five results.

The implementation uses application-side cosine similarity rather than a dedicated database-native vector search engine.

This is technically significant because the application demonstrates the core mechanics of semantic retrieval directly:

- Vector generation

- Vector persistence
- Query vector generation
- Similarity calculation
- Ranking
- Top-K selection

It provides a transparent foundation that can later evolve toward indexed vector infrastructure as the knowledge corpus grows.

Product & Context Engineering

A major part of the platform is the management of AI context.

The application explicitly models several independent context sources:

Conversation History + Personality + Memory + Knowledge



Context Assembly



AI Request

Users can configure whether conversation history, personality, memory and knowledge participate in the conversational workflow.

The conversation model also supports a configurable maximum message-history setting.

This makes context a first-class product capability rather than an implicit side effect of the interface.

Personality & Response Configuration

The platform provides structured personality configuration covering:

- Tone
- Response style
- Formality
- Creativity
- Custom instructions

This configuration is persisted as part of the conversation model, while configurable personality data can also be maintained through frontend application state.

The resulting architecture provides a mechanism for adapting generated responses without changing the underlying application workflow.

Model Configuration

Conversation records include model settings covering:

- Model
- Temperature
- Maximum response tokens

The architecture therefore recognises model configuration as part of the conversational state rather than treating it exclusively as an infrastructure concern.

This creates a foundation for future multi-model orchestration and provider abstraction.

Security Engineering

Security-conscious design is incorporated into the application boundaries.

API credentials

The Groq API credential is accessed through: `process.env.GROQ_API_KEY`

The credential therefore remains on the backend rather than being exposed through frontend application code.

Database configuration

MongoDB connection details are supplied through:

`process.env.MONGODB_URI`

Frontend configuration

The frontend API endpoint is supplied through:

`import.meta.env.VITE_API_URL`

CORS

The Express server defines an explicit allowed-origin configuration including the configured frontend URL and local development origins.

Request validation

The API validates required request structures, including:

- AI message arrays
- Knowledge names
- Knowledge content
- Search queries

File upload controls

Knowledge uploads use:

- Memory storage
- File-size limitation
- Explicit supported file types
- Empty-content validation

Database validation

Mongoose schemas enforce required fields, types, enumerations and validation rules.

Cloud / Development / DevOps

The application is structured for environment-driven deployment.

The backend uses:

```
npm run dev  
→ node --watch src/server.js
```

```
npm start  
→ node src/server.js
```

The frontend uses:

```
npm run dev  
→ Vite development server
```

```
npm run build  
→ Production build
```

```
npm run preview  
→ Production build preview
```

```
npm run lint  
→ Oxlint
```

Environment-specific configuration is separated from application code through environment variables.

The frontend dependency stack is managed through a lockfile using lockfile version 3, while the backend is configured as an ES module Node.js application.

The project is therefore structured around a conventional modern JavaScript full-stack development and deployment workflow.

QA and Engineering Quality

Quality assurance is integrated into the development workflow through several layers.

Static quality

The frontend includes an explicit linting command using **Oxlint**.

Build validation

The frontend includes a production build process through Vite.

Runtime validation

The backend validates:

- API request structures
- Required knowledge data
- Search input

- Uploaded document content
- Vector data

Error handling

The application includes explicit error handling across:

- Database connection
- AI generation
- Streaming responses
- Knowledge processing
- CRUD operations
- File processing
- Frontend API requests

Interactive QA

The product workflow also incorporates:

- Loading states
- Empty states
- Error states
- Toast feedback
- Delete confirmations
- Request cancellation
- Streaming state management

The combination demonstrates QA responsibility extending beyond simply verifying whether an endpoint returns HTTP 200.

Project Management and Product Ownership

The project required decisions beyond writing application code.

Product-level responsibilities included defining:

- Core platform functionality
- Conversation lifecycle
- Knowledge lifecycle
- Memory lifecycle
- Context controls
- Document support
- Retrieval behaviour
- Response behaviour
- Error states
- User workflows
- Future scalability direction

Technical decisions also required balancing implementation complexity against product value.

For example, application-side vector retrieval provides an understandable and relatively lightweight semantic-search architecture without introducing a specialised vector database dependency at the initial architectural stage.

This reflects practical product engineering: selecting technology according to the application's current requirements while preserving a clear path for future evolution.

Maintenance and Engineering Ownership

The platform is structured as an evolving product rather than a single-purpose prototype.

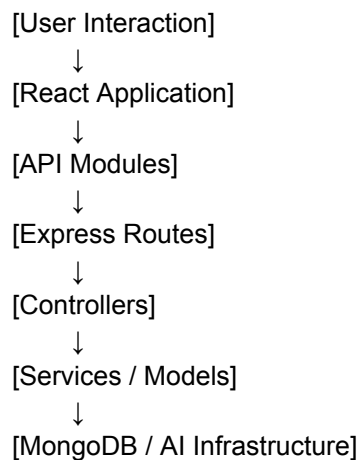
Maintenance responsibilities encompass:

- Frontend dependency management
- Backend dependency management
- API maintenance
- Database model maintenance
- AI provider integration
- Retrieval pipeline maintenance
- Document parser compatibility
- Error handling
- Runtime troubleshooting
- Configuration management
- Build validation
- Linting
- Feature refinement

The modular API, service and model structure allows individual capabilities to evolve without requiring the entire platform to be rewritten.

Engineering Boundaries

The application deliberately separates several concerns:



AI inference is kept behind the backend boundary.

Database access is kept within the backend.

Embedding generation is performed server-side.

Document processing is performed server-side.

The frontend communicates with the platform through HTTP APIs rather than connecting directly to MongoDB or exposing the Groq credential.

This creates a clear separation between client application responsibilities and server-side infrastructure.

Architectural & Engineering Strengths

The strongest engineering characteristics of the platform include:

Full-stack ownership

The project spans the complete application stack from browser execution through API design, server-side processing, database persistence and AI infrastructure.

AI application engineering

The application integrates both generative AI and semantic retrieval rather than relying exclusively on a conventional CRUD architecture.

RAG implementation

The platform incorporates:

document ingestion → chunking → embeddings → vector persistence → semantic search → ranked retrieval

Streaming architecture

The AI pipeline supports progressive generation from the model provider through the Express backend to the React application.

Local embedding infrastructure

Embedding generation uses a local ONNX-compatible model, reducing dependence on an external embedding API.

Persistent application state

Conversations, memories and knowledge remain persistent through MongoDB.

Modular API architecture

Separate routes and controllers exist for the major business resources.

Environment-driven configuration

Infrastructure credentials and deployment-specific configuration are externalised.

Practical engineering trade-offs

The retrieval implementation favours transparency and simplicity over introducing unnecessary infrastructure at the current scale.

Business Impact

The project demonstrates how generative AI can be transformed from a standalone model interaction into an application-level business capability.

Its business value is derived from combining:

Generative AI + persistent context + reusable knowledge + semantic retrieval + application workflows

This combination can support use cases such as:

- Technical knowledge retrieval
- Internal documentation search
- Research assistance
- Personal knowledge management
- Developer productivity
- Domain-specific conversational systems
- Document-driven workflows

The architecture creates an extensible foundation where additional business-specific knowledge, workflows and integrations can be introduced without replacing the fundamental conversational infrastructure.

Technical Impact

From an engineering perspective, the project demonstrates competency across multiple layers of modern software development.

Application engineering

- React
- Vite
- JavaScript / ES Modules
- REST APIs
- State management
- Asynchronous programming

Backend engineering

- Node.js
- Express
- RESTful routing
- Controllers
- Services
- Middleware
- Streaming HTTP responses

Database engineering

- MongoDB
- Mongoose
- Schema design
- Nested documents
- Timestamps
- Validation

- Persistent application state

AI engineering

- Generative AI integration
- LLM inference
- Prompt/context orchestration
- RAG
- Embeddings
- Vector similarity
- ONNX inference
- Hugging Face Transformers

Data processing

- Document ingestion
- PDF extraction
- DOCX extraction
- Text normalisation
- Chunking
- Vector generation
- Ranking

DevOps / engineering operations

- Environment configuration
- NPM scripts
- Build tooling
- Linting
- Development runtime management
- Production execution configuration

Product engineering

- Workflow design
- Feature definition
- Context management
- Error-state design
- Lifecycle management
- Maintenance
- Technical trade-off analysis

Skills Demonstrated

Software Engineering

- Full-stack JavaScript development
- React development
- Node.js development
- Express development
- REST API design
- API integration
- Asynchronous programming
- Streaming architectures
- Modular application design

- Environment configuration
- Error handling
- Runtime troubleshooting

Frontend Development

- React 19
- Vite
- Application state management
- Fetch API
- ReadableStream processing
- AbortController
- Markdown rendering
- Git-style/code syntax rendering
- Responsive application behaviour
- Client-side persistence
- API abstraction

Backend Development

- Node.js
- Express 5
- Middleware
- REST routing
- Controllers
- Service-layer development
- Server-side streaming
- API validation
- File processing
- Multipart upload handling

Database

- MongoDB
- Mongoose
- Schema design
- Nested schemas
- Validation
- Persistent conversation models
- Embedded document structures
- Timestamp management

Generative AI

- LLM integration
- Groq SDK
- Streaming inference
- Context engineering
- Prompt orchestration
- Model configuration
- RAG architecture
- Semantic retrieval
- Embeddings
- Vector similarity

Machine Learning Infrastructure

- Hugging Face Transformers
- ONNX model execution
- Feature extraction
- Mean pooling
- Vector normalisation
- Local embedding inference

Knowledge Engineering

- Document ingestion
- PDF parsing
- DOCX parsing
- Markdown processing
- Text chunking
- Chunk overlap
- Semantic search
- Top-K retrieval
- Similarity scoring

Security

- Server-side API credentials
- Environment variables
- CORS configuration
- Request validation
- File-size controls
- Schema validation

QA

- Linting
- Build validation
- Runtime validation
- API error handling
- Input validation
- Empty-state handling
- Error-state handling
- Request cancellation
- Streaming-state validation

Product & Design

- Product requirements
- Application workflow design
- Information architecture
- UX decision-making
- Feature prioritisation
- Technical trade-off analysis
- Interaction design
- Responsive product design

Engineering Operations

- Dependency management
- NPM tooling
- Development scripts
- Production scripts

- Environment configuration
 - Application maintenance
 - Technical troubleshooting
-

Technology Stack

Frontend

React 19
Vite 8
JavaScript / ES Modules
React Markdown
Remark GFM
Rehype Raw
React Syntax Highlighter
Lucide React

Backend

Node.js
Express 5
CORS
Dotenv
Mongoose

Database

MongoDB

Generative AI

Groq SDK
openai/gpt-oss-20b

Embeddings / Semantic Retrieval

Hugging Face Transformers
ONNX all-MiniLM-L6-v2
Cosine Similarity

Document Processing

Multer
PDF Parse
Mammoth

Engineering Tooling
NPM
Vite
Oxlint
Node.js
Watch Mode

Highlights

The technologies and capabilities that most strongly differentiate this project from a conventional full-stack CRUD application are:

Full-stack AI engineering

The project demonstrates the ability to integrate an LLM into a complete production-oriented application architecture rather than simply calling an AI API from a frontend.

Retrieval-Augmented Generation

The implementation demonstrates practical knowledge of the RAG pipeline from document ingestion through semantic retrieval.

Embedding architecture

The use of a local ONNX embedding model demonstrates understanding of vector representations and semantic retrieval infrastructure.

Streaming

The end-to-end streamed response architecture demonstrates experience with asynchronous data delivery and real-time application behaviour.

API architecture

The platform provides multiple REST resources with dedicated routing and controller boundaries.

Database modelling

The MongoDB schema design supports conversations, nested messages, personality, memory, context, model configuration and embedded knowledge vectors.

Document intelligence

The platform can ingest multiple business-document formats and transform their contents into semantically searchable knowledge.

Product ownership

The project extends beyond implementation into feature definition, workflow design, context controls, user lifecycle management and technical decision-making.

QA and maintenance

The application incorporates validation, error handling, linting, build tooling, cancellation and lifecycle management across the stack.

Areas for Improvement

The current architecture provides several clear opportunities for technical evolution.

Scalable Vector Retrieval

The current application performs similarity comparison in the application layer by loading eligible embedded chunks and calculating cosine similarity.

As the knowledge corpus grows, this could evolve toward:

- MongoDB Atlas Vector Search
- Dedicated vector indexing
- Approximate nearest-neighbour retrieval
- Metadata filtering
- Hybrid keyword + semantic retrieval
- More sophisticated ranking

This would reduce retrieval cost and improve scalability.

Improved Chunking

The current character-based strategy could evolve toward:

- Sentence-aware chunking
- Token-aware chunking
- Paragraph-aware segmentation
- Semantic chunking
- Document-structure-aware chunking

This could improve retrieval precision for complex documents.

Retrieval Context Optimisation

Future retrieval improvements could include:

- Re-ranking
- Similarity thresholds
- Source weighting
- Metadata filtering
- Duplicate-content reduction
- Context-window optimisation

AI Provider Abstraction

The backend could introduce a provider abstraction that allows model selection to be managed consistently rather than coupling generation behaviour to a single provider configuration.

This would support future integration of multiple inference providers and models.

Conversation-Level Security

For a multi-user production deployment, the persistence layer could evolve to include:

- User identities
- Authentication
- Authorisation
- Conversation ownership

- Knowledge ownership
- Memory ownership
- Tenant isolation

Automated Testing

The QA layer could be expanded with:

- Unit tests
- Integration tests
- API tests
- Retrieval-quality tests
- End-to-end browser tests
- AI workflow regression tests

Production Observability

A larger deployment could introduce:

- Structured logging
- Application metrics
- AI latency monitoring
- Retrieval metrics
- Error tracking
- Request tracing
- Usage analytics

These improvements represent natural evolutionary steps from the existing architecture rather than changes to the fundamental product concept.

Professional Role

Role: Full-Stack Software Engineer | AI Application Developer | Product & Technical Owner

The project encompasses responsibilities across the complete software lifecycle:

Product Definition



Architecture



Frontend Engineering



Backend Engineering



Database Engineering



AI Integration



RAG / Knowledge Engineering



Security & Configuration



QA

↓
Maintenance
↓
Future Technical Evolution

My role therefore extended beyond implementing individual features. It covered the technical and product decisions required to turn an AI capability into a coherent full-stack software platform.

Executive Assessment

This project represents a strong demonstration of modern full-stack engineering applied to Generative AI.

Its primary strength is the combination of conventional software engineering with AI-specific infrastructure.

The platform incorporates:

React frontend engineering

- **Node.js / Express API development**
- **MongoDB persistence**
- **document processing**
- **local embedding generation**
- **semantic vector retrieval**
- **RAG context workflows**
- **LLM inference**
- **streamed responses**
- **product and workflow management**

This breadth makes the project particularly relevant to modern software engineering positions involving **AI applications, full-stack development, intelligent systems, knowledge platforms and developer productivity tooling**.

It demonstrates not only the ability to consume AI services, but the ability to engineer the surrounding systems that make AI useful: **data, context, retrieval, persistence, APIs, streaming, validation, product workflows and maintainable application architecture**.
